Drag and drop so simple it hurts http://bevacqua.github.io/dragula/

296 commits 2 branches 58 releases 29 contributors

bevacqua fix slack invite link Latest commit 4ca8359 4 days ago

dist	Release 3.6.3	18 days ago
example	link to slack frame	4 days ago
resources	added link to slack, patreon	4 months ago
test	tests: Remove 'out' event assertions	3 months ago
.editorconfig	initial commit	10 months ago
.gitignore	initial commit	10 months ago
.jshintignore	initial commit	10 months ago
.jshintrc	initial commit	10 months ago
.travis.yml	Add Node.js v4 stable to Travis config	5 months ago
bower.json	Release 3.6.3	18 days ago
changelog.markdown	updated changelog	18 days ago
classes.js	more tests, moved class manipulation to own module	6 months ago
contributing.markdown	slack invite link	4 days ago
dragula.js	Small change allowing dragula.js to work in iframe environments	18 days ago
dragula.styl	unified styles under same selector	6 months ago
favicon.ico	added favicon, logo to landing page	6 months ago
index.html	link to slack frame	4 days ago
license	Update year range to 2016	15 days ago
package.json	Release 3.6.3	18 days ago
readme.markdown	fix slack invite link	4 days ago

readme.markdown

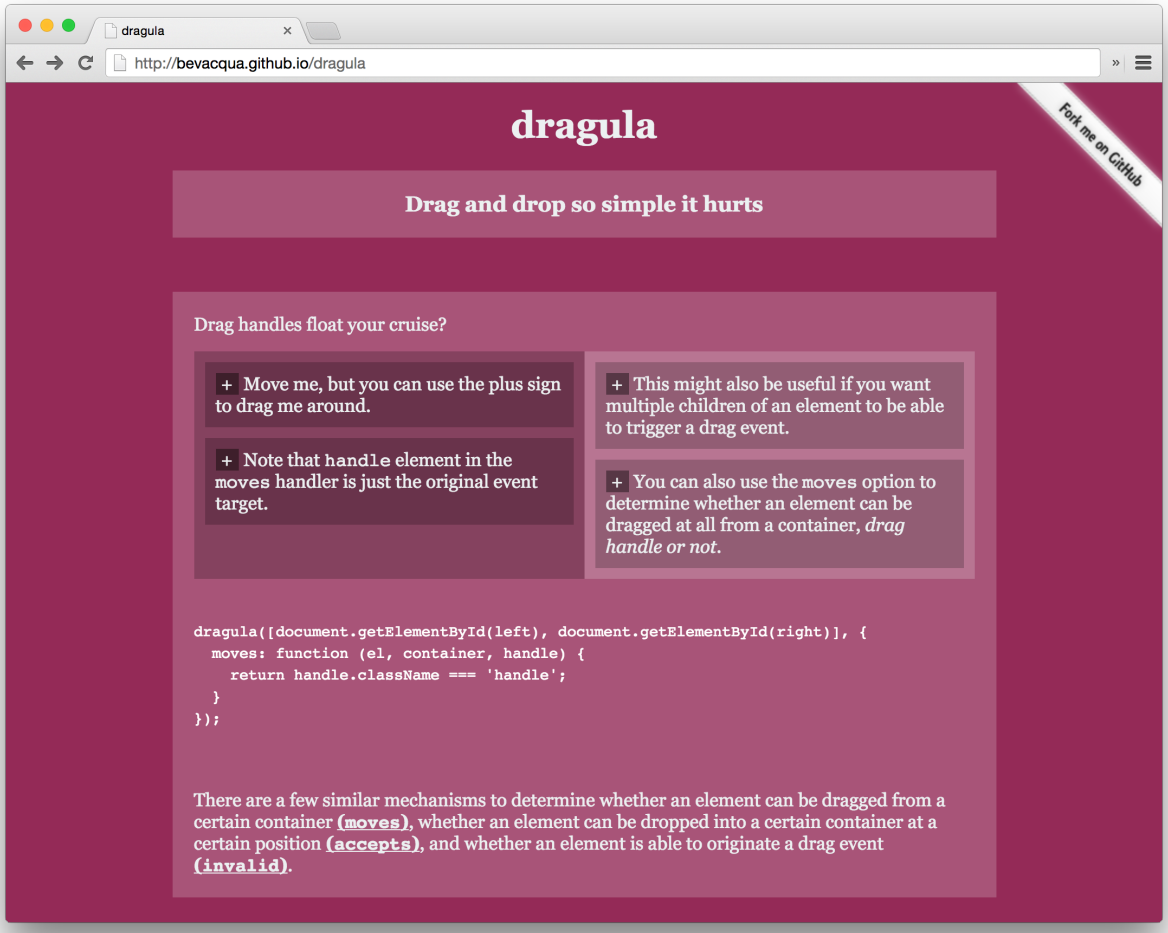
> Drag and drop so simple it hurts

Browser support includes every sane browser and **IE7+**. *(Granted you polyfill the functional* `Array` *methods in ES5)*

Framework support includes vanilla JavaScript, Angular, and React.

- Official Angular bridge for `dragula` *(demo)*
- Official React bridge for `dragula` *(demo)*

Demo



Try out the demo!

Inspiration

Have you ever wanted a drag and drop library that just works? That doesn't just depend on bloated frameworks, that has great support? That actually understands where to place the elements when they are dropped? That doesn't need you to do a zillion things to get it to work? Well, so did I!

Features

- Super easy to set up
- No bloated dependencies
- **Figures out sort order** on its own
- A shadow where the item would be dropped offers **visual feedback**
- Touch events!
- Seamlessly handles clicks *without any configuration*

Install

You can get it on npm.

```
npm install dragula --save
```

Or bower, too.

```
bower install dragula --save
```

If you're not using either package manager, you can use `dragula` by downloading the files in the `dist` folder. We **strongly suggest** using `npm`, though.

Including the JavaScript

There's a caveat to `dragula`. You shouldn't include it in the `<head>` of your web applications. It's bad practice to place scripts in the `<head>`, and as such `dragula` makes no effort to support this use case.

Place `dragula` in the `<body>`, instead.

Including the CSS!

There's a few CSS styles you need to incorporate in order for `dragula` to work as expected.

You can add them by including `dist/dragula.css` or `dist/dragula.min.css` in your document. If you're using Stylus, you can include the styles using the directive below.

```
@import 'node_modules/dragula/dragula'
```

Usage

Dragula provides the easiest possible API to make drag and drop a breeze in your applications.

dragula(containers?, options?)

By default, `dragula` will allow the user to drag an element in any of the `containers` and drop it in any other container in the list. If the element is dropped anywhere that's not one of the `containers`, the event will be gracefully cancelled according to the `revertOnSpill` and `removeOnSpill` options.

Note that dragging is only triggered on left clicks, and only if no meta keys are pressed.

The example below allows the user to drag elements from `left` into `right`, and from `right` into `left`.

```
dragula([document.querySelector('#left'), document.querySelector('#right')]);
```

You can also provide an `options` object. Here's an **overview of the default values**.

```
dragula(containers, {
  isContainer: function (el) {
    return false; // only elements in drake.containers will be taken into account
  },
  moves: function (el, source, handle, sibling) {
    return true; // elements are always draggable by default
  },
  accepts: function (el, target, source, sibling) {
    return true; // elements can be dropped in any of the `containers` by default
  },
  invalid: function (el, target) {
    return false; // don't prevent any drags from initiating by default
```

```
  },
  direction: 'vertical',             // Y axis is considered when determining where an element would be dropped
  copy: false,                       // elements are moved by default, not copied
  copySortSource: false,             // elements in copy-source containers can be reordered
  revertOnSpill: false,              // spilling will put the element back where it was dragged from, if this is true
  removeOnSpill: false,              // spilling will `.remove` the element, if this is true
  mirrorContainer: document.body,    // set the element that gets mirror elements appended
  ignoreInputTextSelection: true     // allows users to select input text, see details below
});
```

You can omit the `containers` argument and add containers dynamically later on.

```
var drake = dragula({
  copy: true
});
drake.containers.push(container);
```

You can also set the `containers` from the `options` object.

```
var drake = dragula({ containers: containers });
```

And you could also not set any arguments, which defaults to a drake without containers and with the default options.

```
var drake = dragula();
```

The options are detailed below.

options.containers

Setting this option is effectively the same as passing the containers in the first argument to `dragula(containers, options)`.

options.isContainer

Besides the containers that you pass to `dragula`, or the containers you dynamically `push` or `unshift` from drake.containers, you can also use this method to specify any sort of logic that defines what is a container for this particular `drake` instance.

The example below dynamically treats all DOM elements with a CSS class of `dragula-container` as dragula containers for this `drake`.

```
var drake = dragula({
  isContainer: function (el) {
    return el.classList.contains('dragula-container');
  }
});
```

options.moves

You can define a `moves` method which will be invoked with `(el, source, handle, sibling)` whenever an element is clicked. If this method returns `false`, a drag event won't begin, and the event won't be prevented either. The `handle` element will be the original click target, which comes in handy to test if that element is an expected *"drag handle"*.

options.accepts

You can set `accepts` to a method with the following signature: `(el, target, source, sibling)`. It'll be called to make sure that an element `el`, that came from container `source`, can be dropped on container `target` before a `sibling` element. The `sibling` can be `null`, which would mean that the element would be placed as the last element in the container. Note that if `options.copy` is set to `true`, `el` will be set to the copy, instead of the originally dragged element.

Also note that **the position where a drag starts is always going to be a valid place where to drop the element**, even if `accepts` returned `false` for all cases.

`options.copy`

If `copy` is set to `true` *(or a method that returns `true`)*, items will be copied rather than moved. This implies the following differences:

Event	Move	Copy
`drag`	Element will be concealed from `source`	Nothing happens
`drop`	Element will be moved into `target`	Element will be cloned into `target`
`remove`	Element will be removed from DOM	Nothing happens
`cancel`	Element will stay in `source`	Nothing happens

If a method is passed, it'll be called whenever an element starts being dragged in order to decide whether it should follow `copy` behavior or not. Consider the following example.

```
copy: function (el, source) {
  return el.className === 'you-may-copy-us';
}
```

`options.copySortSource`

If `copy` is set to `true` *(or a method that returns `true`)* and `copySortSource` is `true` as well, users will be able to sort elements in `copy`-source containers.

```
copy: true,
copySortSource: true
```

`options.revertOnSpill`

By default, spilling an element outside of any containers will move the element back to the *drop position previewed by the feedback shadow*. Setting `revertOnSpill` to `true` will ensure elements dropped outside of any approved containers are moved back to the source element where the drag event began, rather than stay at the *drop position previewed by the feedback shadow*.

`options.removeOnSpill`

By default, spilling an element outside of any containers will move the element back to the *drop position previewed by the feedback shadow*. Setting `removeOnSpill` to `true` will ensure elements dropped outside of any approved containers are removed from the DOM. Note that `remove` events won't fire if `copy` is set to `true`.

`options.direction`

When an element is dropped onto a container, it'll be placed near the point where the mouse was released. If the `direction` is `'vertical'`, the default value, the Y axis will be considered. Otherwise, if the `direction` is `'horizontal'`, the X axis will be considered.

`options.invalid`

You can provide an `invalid` method with a `(el, target)` signature. This method should return `true` for elements that shouldn't trigger a drag. Here's the default implementation, which doesn't prevent any drags.

```
function invalidTarget (el, target) {
  return false;
}
```

Note that `invalid` will be invoked on the DOM element that was clicked and every parent up to immediate children of a `drake` container.

As an example, you could set `invalid` to return `false` whenever the clicked element *(or any of its parents)* is an anchor tag.

```
invalid: function (el) {
  return el.tagName === 'A';
}
```

`options.mirrorContainer`

The DOM element where the mirror element displayed while dragging will be appended to. Defaults to `document.body`.

`options.ignoreInputTextSelection`

When this option is enabled, if the user clicks on an input element the drag won't start until their mouse pointer exits the input. This translates into the user being able to select text in inputs contained inside draggable elements, and still drag the element by moving their mouse outside of the input -- so you get the best of both worlds.

This option is enabled by default. Turn it off by setting it to `false`. If its disabled your users won't be able to select text in inputs within `dragula` containers with their mouse.

API

The `dragula` method returns a tiny object with a concise API. We'll refer to the API returned by `dragula` as `drake`.

`drake.containers`

This property contains the collection of containers that was passed to `dragula` when building this `drake` instance. You can `push` more containers and `splice` old containers at will.

`drake.dragging`

This property will be `true` whenever an element is being dragged.

`drake.start(item)`

Enter drag mode **without a shadow**. This method is most useful when providing complementary keyboard shortcuts to an existing drag and drop solution. Even though a shadow won't be created at first, the user will get one as soon as they click on `item` and start dragging it around. Note that if they click and drag something else, `.end` will be called before picking up the new item.

`drake.end()`

Gracefully end the drag event as if using **the last position marked by the preview shadow** as the drop target. The proper `cancel` or `drop` event will be fired, depending on whether the item was dropped back where it was originally lifted from *(which is essentially a no-op that's treated as a `cancel` event)*.

`drake.cancel(revert)`

If an element managed by `drake` is currently being dragged, this method will gracefully cancel the drag action. You can also pass in `revert` at the method invocation level, effectively producing the same result as if `revertOnSpill` was `true`.

Note that **a *"cancellation"* will result in a `cancel` event** only in the following scenarios.

- `revertOnSpill` is `true`
- Drop target *(as previewed by the feedback shadow)* is the source container **and** the item is dropped in the same position where it was originally dragged from

`drake.remove()`

If an element managed by `drake` is currently being dragged, this method will gracefully remove it from the DOM.

`drake.on` *(Events)*

The `drake` is an event emitter. The following events can be tracked using `drake.on(type, listener)`:

Event Name	Listener Arguments	Event Description
`drag`	`el, source`	`el` was lifted from `source`
`dragend`	`el`	Dragging event for `el` ended with either `cancel`, `remove`, or `drop`
`drop`	`el, target, source, sibling`	`el` was dropped into `target` before a `sibling` element, and originally came from `source`
`cancel`	`el, container, source`	`el` was being dragged but it got nowhere and went back into `container`, its last stable parent; `el` originally came from `source`
`remove`	`el, container, source`	`el` was being dragged but it got nowhere and it was removed from the DOM. Its last stable parent was `container`, and originally came from `source`
`shadow`	`el, container, source`	`el`, *the visual aid shadow*, was moved into `container`. May trigger many times as the position of `el` changes, even within the same `container`; `el` originally came from `source`
`over`	`el, container, source`	`el` is over `container`, and originally came from `source`
`out`	`el, container, source`	`el` was dragged out of `container` or dropped, and originally came from `source`
`cloned`	`clone, original, type`	DOM element `original` was cloned as `clone`, of `type` *(`'mirror'` or `'copy'`)*. Fired for mirror images and when `copy: true`

`drake.destroy()`

Removes all drag and drop events used by `dragula` to manage drag and drop between the `containers`. If `.destroy` is called while an element is being dragged, the drag will be effectively cancelled.

CSS

Dragula uses only four CSS classes. Their purpose is quickly explained below, but you can check `dist/dragula.css` to see the corresponding CSS rules.

- `gu-unselectable` is added to the `mirrorContainer` element when dragging. You can use it to style the `mirrorContainer` while something is being dragged.
- `gu-transit` is added to the source element when its mirror image is dragged. It just adds opacity to it.
- `gu-mirror` is added to the mirror image. It handles fixed positioning and `z-index` *(and removes any prior margins on the element)*. Note that the mirror image is appended to the `mirrorContainer`, not to its initial container. Keep that in mind when styling your elements with nested rules, like `.list .item { padding: 10px; }`.
- `gu-hide` is a helper class to apply `display: none` to an element.

Contributing

See contributing.markdown for details.

Support

There's now a dedicated support channel in Slack. Visit this page to get an invite. Support requests won't be handled through the repository anymore.

License

MIT